

Image Classification on GTSRB using Custom CNN and ResNet-18 models and Regression on CARLA using the PilotNet model

Siddh Sharma

Roll Number: 25135150

Mechanical Engineering

Indian Institute of Technology (BHU), Varanasi

1 Task 0: Traffic Sign Classification using Custom CNN

1.1 CNN Architecture and Discussion

In this I started with basic data preprocessing like normalizing images resizing them and then converting them to tensors. The data was given a 80%,20% split between training and validation, and the test set was given separately.

For designing the architecture I studied different architectures like LeNet-5, AlexNet, VGG-16, ResNet and the purpose and method of using Inception Blocks. I took the following conclusions out of them, initially we want to preserve as much spatial information as we can for low level feature extraction like edges and corners. Then as we move along the CNN we want to reduce the dimensions of height and width and increase the number of channels (i.e. reduce n_h , n_w and increase n_c). A good architecture is the one which has an appropriate number of parameters which balances accuracy and computation. Also architectures which are more uniform in their CNN layers often perform better than uneven models. I learnt that Inception Blocks can be used when we are unsure about which kernel size to take and if we want to add pooling or not as it uses all of them and presents a concatenated result. I also realized that very deep models are not viable without using Residual Blocks as without Residual Blocks their training error after a point starts increasing as they become more deep. Residual Blocks give us a way to use deep neural networks without increasing the training error and often help in increasing the accuracy.

Following these principles I made my own Custom CNN architecture.

My model consists of 18 layers. The network takes a $128 \times 128 \times 3$ input and passes it through 32 filters of size 3×3 and same padding. The kernel size was taken so as it captures enough spatial information for low level feature extraction despite being small and thus also reduces computation time. 32 filters were used as we want to start with a small amount as initially we only use it for low level feature extraction which should suffice with 32 filters. The padding was taken as we initially wanted to preserve as much spatial information as possible. We then pass the output through the same layer again. We used these 2 layers for low level feature extraction.

After passing through the first 2 layers the output of shape $128 \times 128 \times 32$ is passed through a MaxPooling layer before the Inception Block, so as to reduce the spatial dimension and decrease the computation time in the Inception Block.

The Inception Block consists of kernel sizes of 3 with 128 filters (same padding) and 5 with 64 filters (same padding) and a MaxPooling layer with padding and stride of 1 which is later passed through a $1 \times 1 \times 64$ layer. The final output of this is the concatenation of the outputs of the 3 layers with a size of $64 \times 64 \times 256$.

Since the Inception Block is used here for mid-level feature extraction we used kernels of sizes 3 and 5. The number of filters of 3 was more than 5 as we want to emphasize the smaller features more. The method of bottleneck was

used in computation of both kernels 3 and 5 as it helps in reducing computation time.

The block was then passed through a MaxPooling layer and becomes of the size $32 \times 32 \times 256$. This was done to reduce the computation time in the next Inception Block.

The output is then passed through the same Inception Block again giving an output of $32 \times 32 \times 256$. These 2 Inception Blocks were used for mid-level feature extraction.

The final part of the convolution network consists of 2 Residual Blocks. As the network is now getting deeper, Residual Blocks are introduced to help in the high level feature extraction without drop in accuracy.

The first Residual Block takes an input of $32 \times 32 \times 256$ and uses downsampling as well to return an output of $16 \times 16 \times 512$ and after passing through the second Residual Block the shape stays the same as $16 \times 16 \times 512$ as there is no downsampling in the second block.

The Residual Blocks are the same as the last residual block combination in ResNet-18 with the same structure. The same block combination was used as it worked well in ResNet-18 and my output channels before the Residual Blocks were the same so it made sense to use the same structure.

The output of $16 \times 16 \times 512$ is passed through a Global Average Pooling Layer and converted to $1 \times 1 \times 512$.

GAP is taken as by the time we reach the last layer all of the high level features are extracted so the spatial location of that feature is not that important but rather how much of that feature is present in the image is important. GAP tells us exactly that for all high level features.

In all of the CNN layers BatchNorm and ReLU were implemented on the outputs in all the 16 layers. ReLU activation was used to introduce non-linearity and BatchNorm was introduced as it normalizes activation and thus helps in higher learning rates and faster convergence. It also prevents vanishing and exploding gradients.

The last part consists of FCL. The output from the GAP is taken as the first layer with 512 neurons. The hidden layer consists of 512 neurons with Dropout, BatchNorm and ReLU activation and the last layer consists of 43 neurons which is the output layer. The hidden layer consists of BatchNorm and ReLU for the same purposes stated above while Dropout was used to prevent overfitting.

1.2 Performance Metrics

Table 1: Performance Metrics of the Custom CNN on the GTSRB Dataset

Metric	Value
Accuracy	99.00%
Precision	99.03%
Recall	99.00%
F1 Score	99.00%

1.2.1 Confusion Matrix

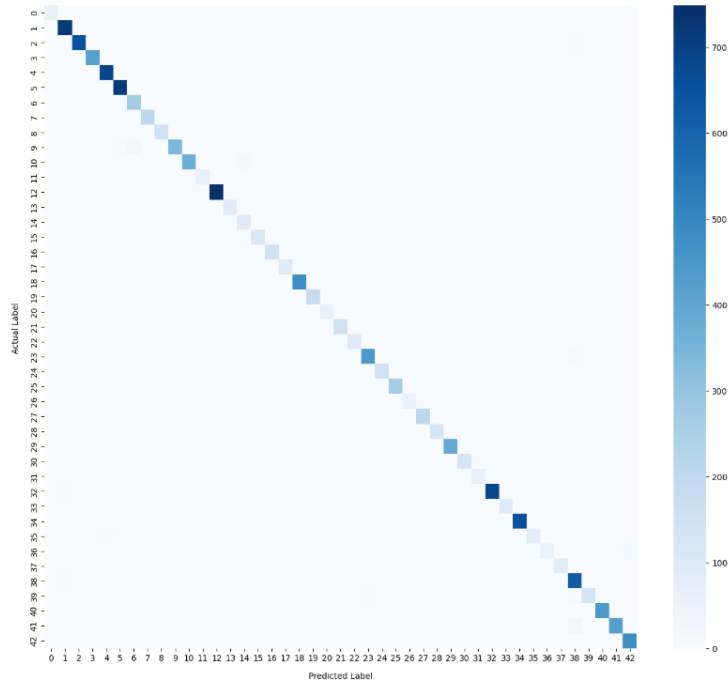


Figure 1: Confusion Matrix of the Custom CNN on the GTSRB Dataset

1.2.2 Training and Validation Loss Curves

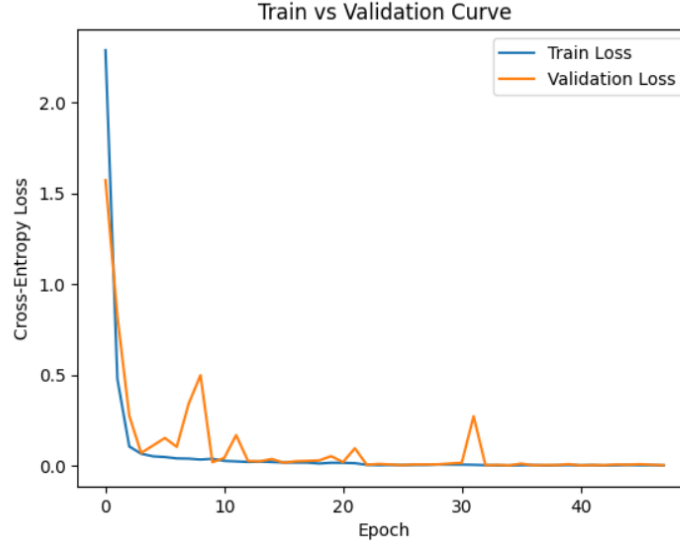


Figure 2: Training and Validation Loss Curves

1.3 Discussion and Analysis

The Custom CNN was able to get an excellent classification performance on the GTSRB dataset with an accuracy of 99.00%, precision of 99.03%, recall of 99.00%, and an F1-score of 99.00%. This goes on to show that the model correctly identified a vast majority of traffic signs while also getting a good balance between precision and recall.

The training and validation loss curves show rapid convergence during the initial epochs, which was followed by stable curves with very little loss. However there are a few fluctuations in the validation loss but the overall graph shows it following the training loss, thus proving good generalization and not much overfitting.

The confusion matrix shows darkness along the diagonal, showing that most of the samples were classified correctly. Very little misclassifications are shown in certain classes, which was expected as there are similarities between some traffic signs. Overall, the results show that the Custom CNN performed very well for traffic sign recognition and is able to classify most of them correctly.

2 Task 1: Steering Angle Prediction using PilotNet

2.1 PilotNet Architecture and Discussion

This task is of steering angle prediction which is a regression problem rather than a classification problem. Instead of predicting one of out of many classes, the network must learn and predict a continuous steering angle value from the input image provided to it. Also, the loss function used in the model is Mean Squared Error (MSE) Loss during training and Root Mean Squared Error (RMSE) for evaluation.

I used a couple of data augmentation techniques during training to improve the learning of the model. Most augmentations such as brightness adjustments and image transformations did not involve changing the steering angle labels. However horizontal flipping required a change in the label as we are flipping the image so the steering angle label was changed to its negative value.

A difficulty in the dataset was the imbalance in steering angles as most of the samples had near zero steering angle labels as the vehicle might have been driving straight for a large portion of the time the data was being collected. My first attempt without any weighted sample balancing led to the model predicting values very close to zero for a majority of test data. To fix this, weighted sampling was used in training to increase the number of less frequent steering angles in the mini-batches. Although this led to a slight increase in RMSE, the model learned to predict more meaningful steering values instead of predicting near zero values for all samples at the time of evaluation.

2.1.1 Architecture

The PilotNet architecture has been made the same as described in the paper from scratch.

The network takes an input image of size $66 \times 200 \times 3$. The convolutional part of the network consists of five convolutional layers:

- Convolution Layer with 24 filters of size 5×5 and stride 2
- Convolution Layer with 36 filters of size 5×5 and stride 2
- Convolution Layer with 48 filters of size 5×5 and stride 2
- Convolution Layer with 64 filters of size 3×3 and stride 1
- Convolution Layer with 64 filters of size 3×3 and stride 1

A ReLU activation function is applied after each convolutional layer. The first three convolutional layers are responsible for reducing the spatial dimensions of the image while extracting low-level and mid-level features. The final two convolutional layers perform high-level feature extraction required for steering angle prediction. The output of the final convolutional layer has dimensions

$64 \times 1 \times 18$. This output is then flattened into a vector of 1152 features and passed through the Fully Connected Layers consisting of:

- 100 neurons
- 50 neurons
- 10 neurons
- 1 output neuron

ReLU activation is applied after each hidden layer. The final layer consists of a single neuron which predicts the steering angle.

2.2 Performance Metrics

2.2.1 Root Mean Squared Error (RMSE)

Weather Condition	RMSE
ClearNoon	0.02871
ClearSunset	0.04764
CloudyNoon	0.02339
CloudySunset	0.02481
HardRainNoon	0.02220
HardRainSunset	0.02699
MidRainSunset	0.02914
MidRainyNoon	0.02277
SoftRainNoon	0.02202
SoftRainSunset	0.02531
WetCloudyNoon	0.02284
WetCloudySunset	0.02599
WetNoon	0.02891
WetSunset	0.03895

Table 2: PilotNet’s Test Set RMSEs Across Different Weather Conditions

2.2.2 Training and Validation RMSE Curves

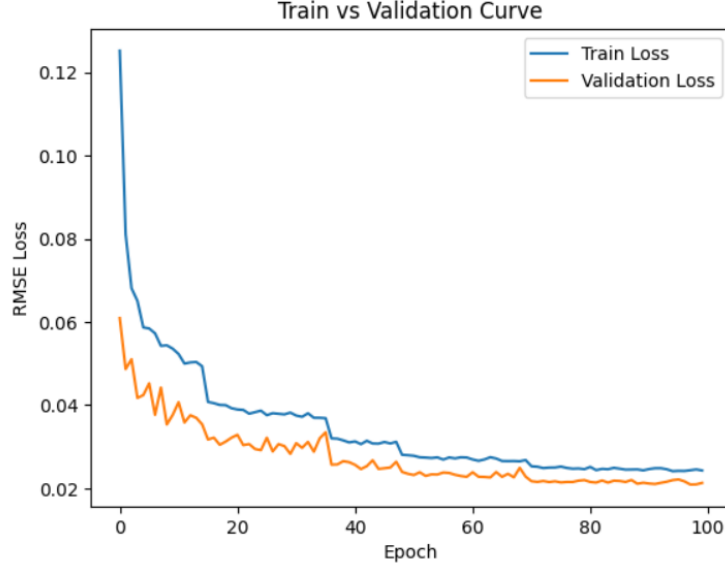


Figure 3: Training and Validation RMSE Curves for PilotNet

2.3 Discussion and Analysis

The PilotNet model got low RMSE values in all weather conditions, showing its ability to learn the relationship between road images and steering commands. The training and validation RMSE curves show roughly steady decreasing plots throughout training, showing good learning and not much overfitting as the validation RMSE is close to the training RMSE in the entire training session.

The model got its lowest errors in weathers like SoftRainNoon, HardRainNoon, and MidRainyNoon, while the highest errors were seen in ClearSunset and WetSunset conditions. This might be because during sunset there are lighting variations, shadows and also glare making it difficult for the model to identify discriminative road features. Also wet road conditions show reflections that can change the appearance of lane markings and road boundaries making the model more confused and increased prediction error.

3 Task 2: Traffic Sign Classification using ResNet-18

3.1 ResNet-18 Architecture and Discussion

The residual connection is a way information from the previous layers is directly passed to the layers ahead through a shortcut path by just adding the input to the residual block to the output of the next layer. Thus if the output of the network is $(F(x))$ the residual output $(H(x))$ becomes $(F(x)+x)$, where (x) is the input of the residual block.

$$H(x) = F(x) + x$$

This helps as if a deeper layer learns nothing new the original input is already available to be taken as output through the shortcut and it becomes an identity mapping as $(F(x))$ becomes zero. In absence of the residual layer the layers would still have to learn the identity mapping, thus the residual network helps record only the changes which take place whereas plain networks have to map the original information again even if there is no new information.

Thus Residual Networks solve the problem of deeper plain networks having higher training error than shallow networks (degradation). Also Residual Networks help reduce the problem of vanishing gradients by allowing the gradient to pass through the shortcut connections, the shortcut also helps to pass important features from earlier layers to deeper layers. Also it provides a passage for earlier features to be passed down to the deeper layers thus helping save important information.

Thus it helps in making deeper networks without increasing training loss. Theoretically deeper models should perform at least as well as the shallow ones but that is not the case. This is the problem Residual Networks solve, making networks easier to optimize and train by making it easier for layers to learn identity mappings when no new information is added to them.

3.1.1 Architecture

The ResNet-18 architecture is the same as it was described in the paper and was implemented from scratch.

The architecture start with a 7×7 convolution layer with 64 filters and stride 2 and then it passed through a MaxPooling layer. Then it is passed through 4 combinations of 2 Residual Blocks each:

- Two Residual Blocks with 64 channels
- Two Residual Blocks with 128 channels
- Two Residual Blocks with 256 channels
- Two Residual Blocks with 512 channels

Each Residual Block is of the following structure:

- 3×3 Convolution
- Batch Normalization
- ReLU Activation
- 3×3 Convolution
- Batch Normalization
- Residual Addition
- ReLU Activation

Whenever there is a change in the spatial dimensions, downsampling is used through the shortcut connection to make sure there are matching dimensions before addition.

After the final residual block, the output is passed through Global Average Pooling and then through a Fully Connected Layer for classification on the GTSRB dataset.

3.2 Performance Metrics

Table 3: Performance Metrics of ResNet-18 on the GTSRB Dataset

Metric	Value
Accuracy	98.01%
Precision	98.09%
Recall	98.01%
F1 Score	97.98%

3.2.1 Confusion Matrix

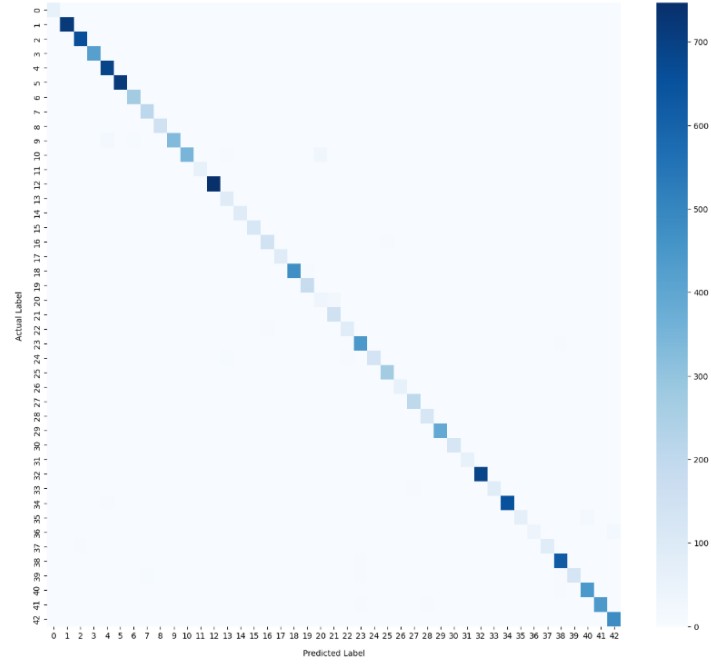


Figure 4: Confusion Matrix of ReNet-18 on the GTSRB Dataset

3.2.2 Training and Validation Loss Curves

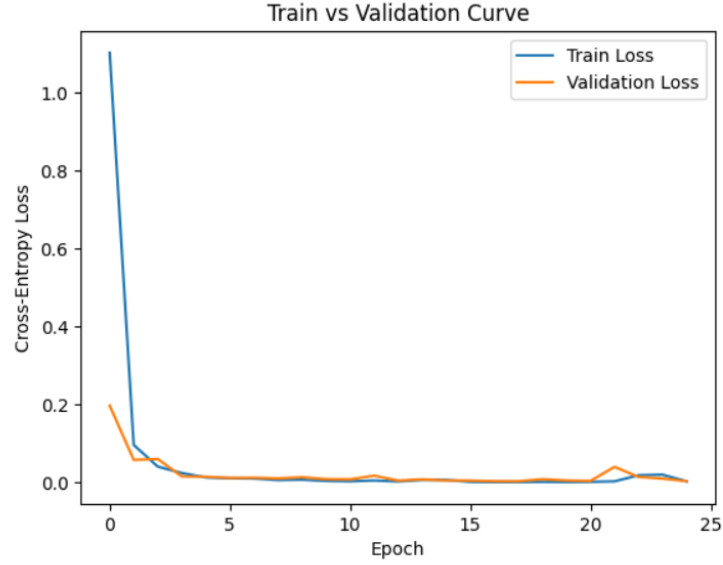


Figure 5: Training and Validation Loss Curves

3.3 Discussion and Analysis

The ResNet-18 model also showed a strong performance on the GTSRB dataset with an accuracy of 98.01%, precision of 98.09%, recall of 98.01%, and an F1-score of 97.99%. The training and validation loss curves show converge rapidly in the beginning epochs and then stable curves are seen with not much overfitting. The confusion matrix is darker along diagonal entries, showing that most of the traffic signs were classified correctly, and only a few were misclassified due to some traffic signs being very similar visually.

As compared to my Custom CNN, ResNet-18 got slightly lower classification metric results. However, ResNet-18 took significantly less computational time due to its architecture design as it works on very perfectly defined residual blocks. Overall, both models performed well on the dataset, with the Custom CNN showing higher accuracy while ResNet-18 took less computational time while showing slightly less accuracy.

4 Conclusion

In this assignment, CNN models were made and evaluated for both the image classification task and the steering angle prediction task. The Custom CNN and ResNet-18 models got high classification results on the GTSRB dataset, showing the ability of CNNs for traffic sign recognition. The Custom CNN got the best overall performance, while ResNet-18 had a simpler architecture getting slightly less accuracy with lower computational time.

For the Carla dataset, the PilotNet architecture learned how to do steering angle predictions and made predictions in a variety of weather conditions. Despite its small size, the network got low RMSE values and showed its ability to learn discriminative features directly from images.

Overall, through the assignment I got a practical experience in designing, training, and evaluating CNNs for both classification and regression problems, while learning the importance of architectural design and data preprocessing,